

S&DS 365 / 665
Intermediate Machine Learning

The Neural Tangent Kernel and Gaussian Processes

October 5

Yale

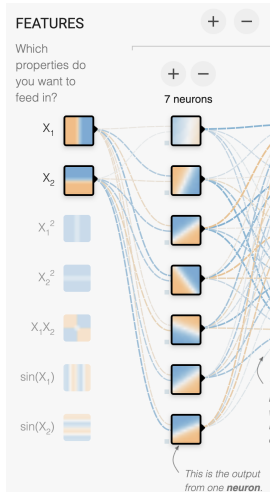
Reminders

- Assignment 2 is out
- Quiz 3 Wednesday, online
- Exam #1 Monday October 19, in class

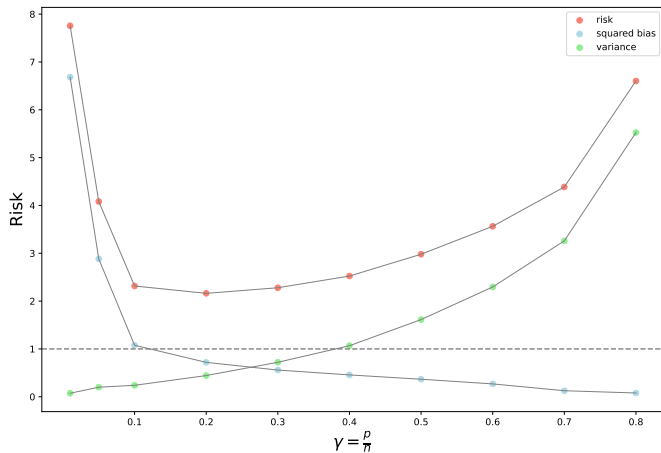
For Today and Wednesday

- Recap of double descent
- The neural tangent kernel (NTK)
- Recap of Gaussian Processes
- Connection between GPs and NNs
- Need for approximate inference
- Variational autoencoders (VAEs)

A fruit fly for deep learning: Random features



Classical risk: Single descent for OLS



Double descent

We went over notes on the double descent phenomenon on the board. Be sure you go through them in detail!

<https://github.com/YData123/sds365-fa26/raw/main/notes/double-descent.pdf>

OLS and minimal norm solution

OLS: $p < n$

$$\hat{\beta} = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{Y}$$

Minimal norm solution: $p > n$:

$$\hat{\beta}_{\text{mn}} = \mathbb{X}^T (\mathbb{X} \mathbb{X}^T)^{-1} \mathbf{Y}$$

“Ridgeless regression”

As λ decreases to zero, the ridge regression estimate:

- Converges to OLS in the “classical regime” $\gamma < 1$
- Converges to $\hat{\beta}_{mn}$ in “overparameterized regime” $\gamma > 1$

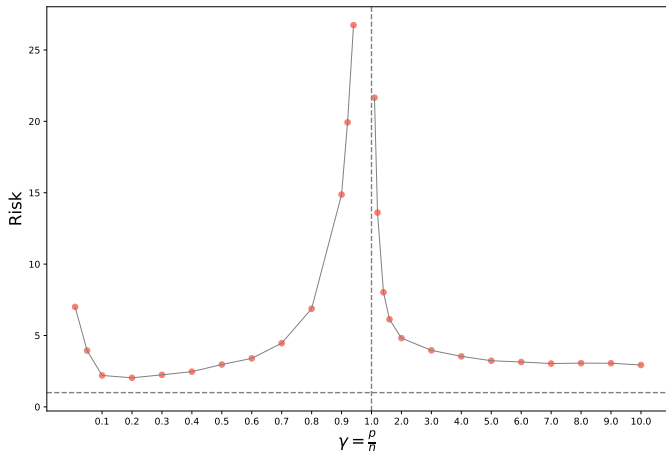
“Ridgeless regression”

This is a consequence of

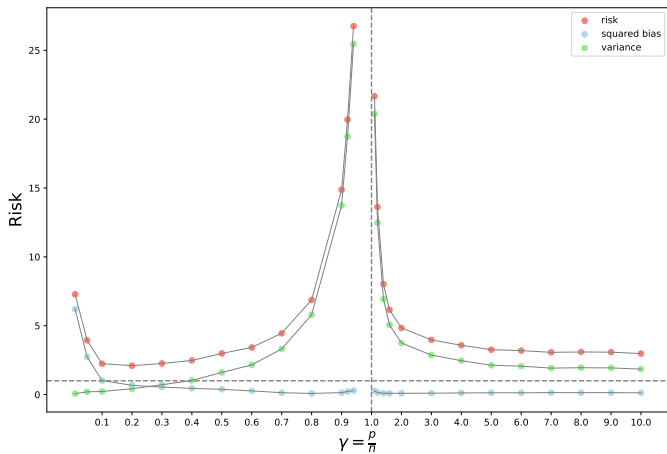
$$(X^T X + \lambda I_p)^{-1} X^T = X^T (X X^T + \lambda I_n)^{-1}$$

which follows from the Woodbury formula (Assn 2).

Double descent



Double descent



Double descent

- As $\gamma \rightarrow \infty$, the bias stays small – data are always interpolated
- Each entry of $\frac{1}{p}\mathbf{X}\mathbf{X}^T$ is the average over an increasing number of identically distributed random vectors
- As a result, the variance decreases

Double descent in LLMs

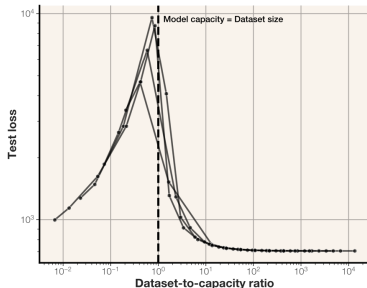


Figure 3 In our experiments on synthetic bitstrings, double descent occurs exactly when the dataset size begins to exceed the model's capacity, when unintended memorization is no longer beneficial for lowering the loss.

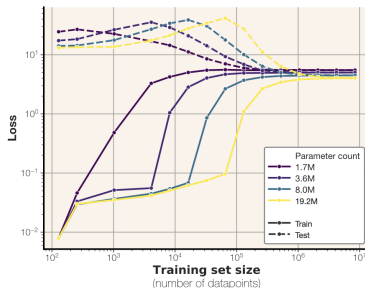


Figure 4 Train and test losses of different model and dataset sizes trained on text. Double descent occurs when dataset size exceeds model capacity.

Neural tangent kernel

The *neural tangent kernel (NTK)* has been useful in understanding the performance of large neural networks, and the dynamics of stochastic gradient descent training.

Parameterized functions

Suppose we have a parameterized function $f_{\theta}(x) \equiv f(x; \theta)$

Almost all machine learning takes this form — for classification and regression, these give us estimates of the regression function

For neural nets, the parameters θ are all of the weight matrices and bias (intercept) vectors across the layers.

Feature maps

Suppose we have a parameterized function $f_\theta(x) \equiv f(x; \theta)$

We then define a *feature map*

$$x \mapsto \varphi(x) = \nabla_\theta f(x; \theta_0) = \begin{pmatrix} \frac{\partial f(x; \theta_0)}{\partial \theta_1} \\ \frac{\partial f(x; \theta_0)}{\partial \theta_2} \\ \vdots \\ \frac{\partial f(x; \theta_0)}{\partial \theta_p} \end{pmatrix}$$

This defines a Mercer kernel called the *neural tangent kernel (NTK)*

$$K(x, x') = \varphi(x)^T \varphi(x') = \nabla_\theta f(x; \theta_0)^T \nabla_\theta f(x'; \theta_0)$$

Feature maps

This defines a Mercer kernel

$$K(x, x') = \varphi(x)^T \varphi(x') = \nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0)$$

What is the NTK for the random features model?

Feature maps

The NTK for the random features model is

$$K(x, x') = h(x)^T h(x')$$

Feature maps

Conversely, a deep neural network with a large number of neurons is approximately equivalent to a random features model!

Why? The next three slides sketch the argument.

NTK and SGD

- The dynamics of stochastic gradient descent for deep networks has been studied
- Mathematical result: As the number of neurons in the layers grows, the parameters in the network barely change during training, even though the training error quickly decreases to zero

NTK and random features

Consequence: If the parameters only change by a small amount, a linear approximation can be used:

Let $\theta = \theta_0 + \delta$. Then

$$\begin{aligned} f(x, \theta) &\approx f(x, \theta_0) + \nabla_{\theta} f(x, \theta_0)^T \delta \\ &= \nabla_{\theta} f(x, \theta_0)^T \delta \end{aligned}$$

assuming that $f(x, \theta_0) = 0$ (not a problem to assume this)

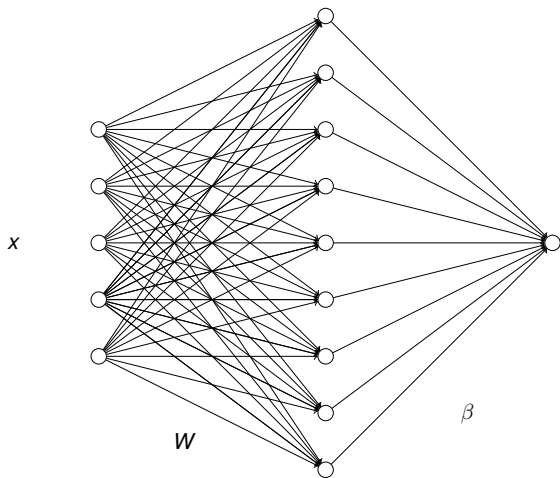
Note: δ here is not the vector of weights in the last layer. It's a vector of new parameters that combine the "features" that are the derivatives of the neural net output as a function, with respect to the weights, evaluated at a random initialization.

NTK and random features

Putting these two results together, tells us that a neural network is (approximately) equivalent to a random features model!

The random features are $h(x) \equiv \nabla_{\theta} f(x, \theta_0)$

Dimension of $\nabla_{\theta} f(x, \theta_0)$?



Summary so far

- Insight into properties: Overparameterization and double descent
- First kernel connection: Neural Tangent Kernel (NTK)
- Another fundamental kernel connection is coming!

Recap of Gaussian Processes

Bayesian Inference

The parameter θ of a model is viewed as a random variable.
Inference usually carried out as follows:

- Choose a *generative model* $p(x | \theta)$ for the data.
- Choose a *prior distribution* $\pi(\theta)$ that expresses beliefs about the parameter before seeing any data.
- After observing data $\mathcal{D}_n = \{x_1, \dots, x_n\}$, update beliefs and calculate the *posterior distribution* $p(\theta | \mathcal{D}_n)$.

Bayes' Theorem

A simple consequence of conditional probability:

$$\begin{aligned}\mathbb{P}(A|B) &= \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \\ &= \frac{\mathbb{P}(B|A) \mathbb{P}(A)}{\mathbb{P}(B)}\end{aligned}$$

Bayes' Theorem

The posterior distribution can be written as

$$p(\theta | x_1, \dots, x_n) = \frac{p(x_1, \dots, x_n | \theta)\pi(\theta)}{p(x_1, \dots, x_n)} = \frac{\mathcal{L}_n(\theta)\pi(\theta)}{c_n} \propto \mathcal{L}_n(\theta)\pi(\theta)$$

where $\mathcal{L}_n(\theta)$ is the *likelihood function* and

$$c_n = p(x_1, \dots, x_n) = \int p(x_1, \dots, x_n | \theta)\pi(\theta)d\theta = \int \mathcal{L}_n(\theta)\pi(\theta)d\theta$$

is the normalizing constant, which is also called *evidence*.

Nonparametric Bayes

- Bayesian inference for infinite-dimensional spaces
- Alternative to classical/frequentist approaches
- Caution required with interpretation

Nonparametric Bayes

- In nonparametric Bayesian inference, we replace a finite dimensional model θ with an infinite dimensional model
- This is usually a class of *functions*
- Typically neither the prior nor the posterior have a density; but the posterior is still well defined.

Key points

- In a Bayesian approach, the parameters are random, and the data are fixed.
- In nonparametric Bayes, the “parameters” are functions
- A Gaussian process is a stochastic process m where each collection of random variables $m(x_1), m(x_2), \dots, m(x_n)$ is jointly Gaussian
- Gaussian processes are Bayesian versions of kernel regression; the posterior mean is equivalent to Mercer kernel regression

Stochastic processes

A stochastic process is a collection of random variables indexed some set (such as time), all defined with respect to a common probability space.

We'll focus on a fundamental stochastic process: The Gaussian process

More technically, a stochastic process $\{X(t)\}_{t \in T}$ is a collection of random variables indexed by a set T and defined on a common probability space $(\Omega, \mathcal{F}, P)$ where Ω is a sample space, $\mathcal{F}$ is a σ -algebra, and P is a probability measure.

Gaussian processes

The nonparametric regression model is

$$Y_i = m(X_i) + \epsilon_i, \quad i = 1, \dots, n$$

where $\mathbb{E}(\epsilon_i) = 0$.

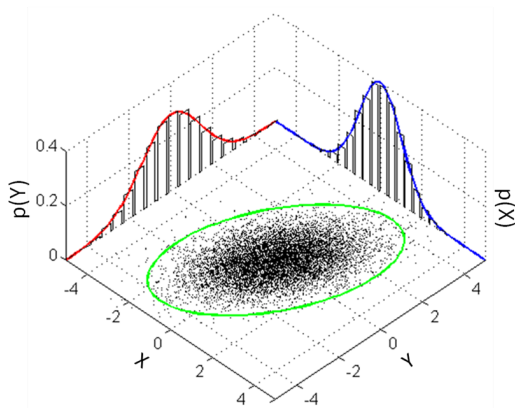
The frequentist kernel estimator for m is

$$\hat{m}(x) = \frac{\sum_{i=1}^n Y_i K\left(\frac{x-X_i}{h}\right)}{\sum_{i=1}^n K\left(\frac{x-X_i}{h}\right)}$$

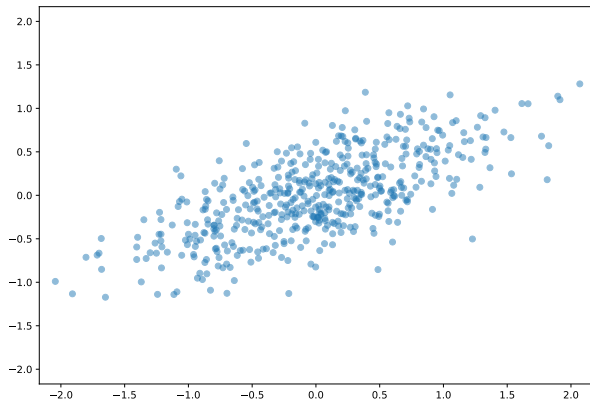
where K is a kernel and h is a bandwidth.

Bayesian version requires prior π on set of regression functions

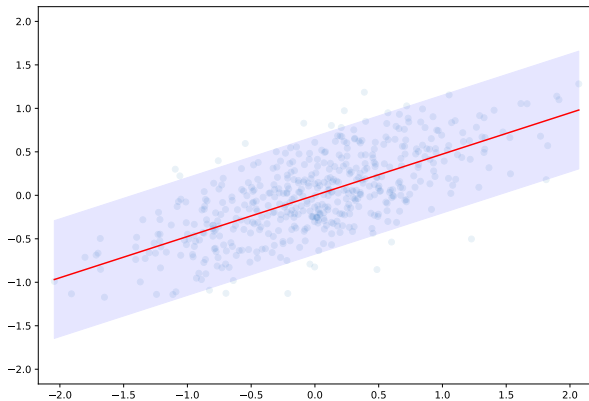
Gaussian marginals and conditionals



Starting point: Conditionals of Gaussian



Starting point: Conditionals of Gaussian



Gaussian conditionals

If (X_1, X_2) are jointly Gaussian with distribution

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim N \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix} \right)$$

then the conditional distributions are also Gaussian and given by

$$X_1 | x_2 \sim N \left(\frac{K_{12}}{K_{22}} x_2, K_{11} - \frac{K_{12}^2}{K_{22}} \right)$$

$$X_2 | x_1 \sim N \left(\frac{K_{12}}{K_{11}} x_1, K_{22} - \frac{K_{12}^2}{K_{11}} \right)$$

Gaussian process

A stochastic process $m(x)$ indexed by $x \in \mathbb{R}$ is a *Gaussian process* if for each set of points $x_1, \dots, x_n$ the vector $(m(x_1), m(x_2), \dots, m(x_n))^T$ is normally distributed:

$$(m(x_1), m(x_2), \dots, m(x_n))^T \sim N(\mu(x), K(x))$$

where $\mu(x) = (\mu(x_1), \dots, \mu(x_n))$ and $K_{ij}(x) = K(x_i, x_j)$ is a Mercer kernel.

As before, if $x_1, \dots, x_n$ are fixed we denote the $n \times n$ matrix with entries $K(x_i, x_j)$ by $\mathbb{K}$.

Gaussian process prior

Let's assume $\mu = 0$, so prior mean function is zero

Density of the Gaussian process prior of $m = (m(x_1), \dots, m(x_n))$ is

$$\pi(m) = (2\pi)^{-n/2} |\mathbb{K}|^{-1/2} \exp \left(-\frac{1}{2} m^T \mathbb{K}^{-1} m \right).$$

Under change of variables $m = \mathbb{K}\alpha$, we have $\alpha \sim N(0, \mathbb{K}^{-1})$ and

$$\pi(\alpha) = (2\pi)^{-n/2} |\mathbb{K}|^{1/2} \exp \left(-\frac{1}{2} \alpha^T \mathbb{K} \alpha \right).$$

Gaussian processes prior

What functions have high probability according to the Gaussian process prior?

The prior favors $m^T \mathbb{K}^{-1} m$ being small. If v is an eigenvector of $\mathbb{K}$, with eigenvalue λ , then

$$\frac{1}{\lambda} = v^T \mathbb{K}^{-1} v$$

- Eigenfunctions of the Mercer kernel K with *large* eigenvalues are favored by the prior
- These correspond to smooth functions; the eigenfunctions that are very wiggly correspond to small eigenvalues

Using the likelihood

We observe $Y_i = m(x_i) + \epsilon_i$ where $\epsilon_i \sim N(0, \sigma^2)$. So, log-likelihood is

$$\log p(Y | m) = -\frac{1}{2\sigma^2} \sum_i (Y_i - m(x_i))^2 + C$$

where $C = -\log(\sqrt{2\pi\sigma^2})$.

Log-posterior is

$$\begin{aligned}\log p(Y | m) + \log \pi(m) &= -\frac{1}{2\sigma^2} \|Y - \mathbb{K}\alpha\|_2^2 - \frac{1}{2} \alpha^T \mathbb{K}\alpha + C' \\ &= -\frac{1}{2\sigma^2} \|Y - \mathbb{K}\alpha\|_2^2 - \frac{1}{2} \|\alpha\|_K^2 + C'\end{aligned}$$

C' is just another constant.

Calculating the posterior

In Bayesian *maximum a posteriori (MAP)* inference, one estimates the mode of the posterior.

The posterior mean (and mode) is

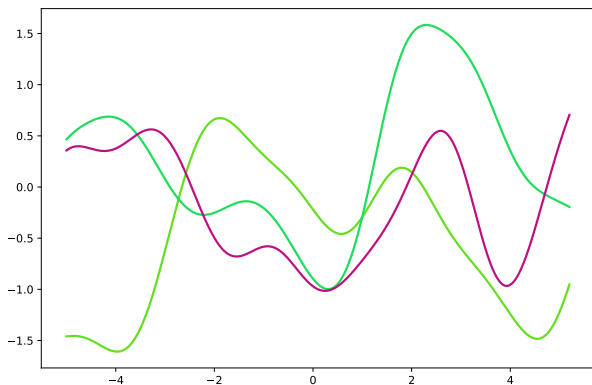
$$\mathbb{E}(\alpha \mid Y) = \left(\mathbb{K} + \sigma^2 I \right)^{-1} Y$$

and thus

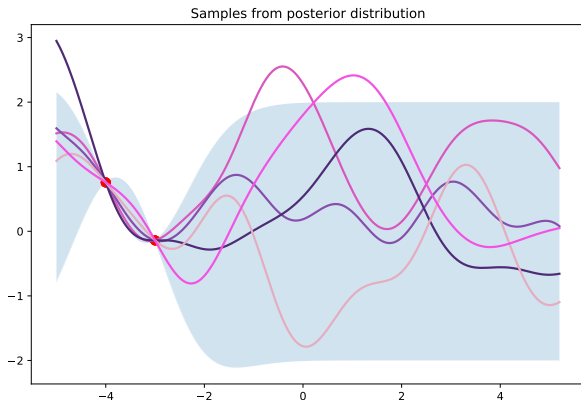
$$\hat{m} = \mathbb{E}(m \mid Y) = \mathbb{K} \left(\mathbb{K} + \sigma^2 I \right)^{-1} Y.$$

Equivalent to Mercer kernel regression

Samples from prior and posterior



Samples from prior and posterior



Gaussian conditionals

If (X_1, X_2) are jointly Gaussian with distribution

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \sim N \left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} A & C \\ C^T & B \end{pmatrix} \right)$$

then the conditional distributions are also Gaussian and given by

$$X_1 | x_2 \sim N \left(\mu_1 + CB^{-1}(x_2 - \mu_2), A - CB^{-1}C^T \right)$$

$$X_2 | x_1 \sim N \left(\mu_2 + C^T A^{-1}(x_1 - \mu_1), B - C^T A^{-1}C \right)$$

Predicting at a new point

How do we predict $Y_{n+1} = m(x_{n+1}) + \epsilon_{n+1}$?

Let k be the vector

$$k = (K(x_1, x_{n+1}), \dots, K(x_n, x_{n+1})).$$

Then $(Y_1, \dots, Y_{n+1})$ are jointly Gaussian with covariance

$$\begin{pmatrix} \mathbb{K} + \sigma^2 I & k \\ k^T & K(x_{n+1}, x_{n+1}) + \sigma^2 \end{pmatrix}.$$

Predictive distribution

Using above expression for Gaussian conditionals:

The posterior mean and variance are

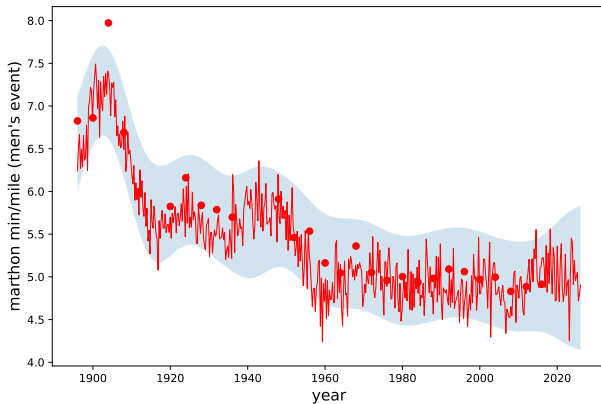
$$\mathbb{E}(Y_{n+1} \mid x_{1:n}, Y_{1:n}) = k^T (\mathbb{K} + \sigma^2 I)^{-1} Y$$

$$\text{Var}(Y_{n+1} \mid x_{1:n}, Y_{1:n}) = K(x_{n+1}, x_{n+1}) + \sigma^2 - k^T (\mathbb{K} + \sigma^2 I)^{-1} k$$

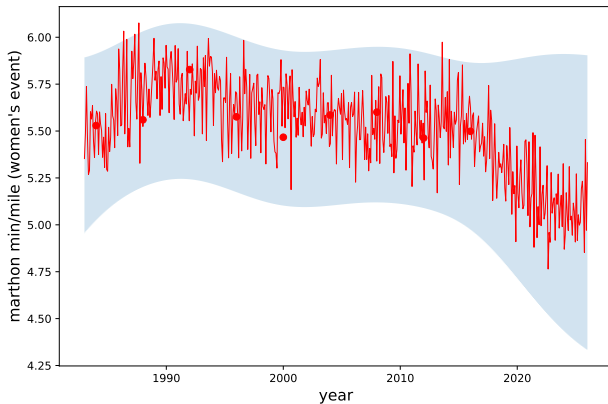
Predictive distribution

- Note that the mean is identical to what we saw for Mercer kernel regression
- But now we get a measure of uncertainty (the variance), which comes from the Gaussian process assumption

Olympic marathon times (men's race)



Olympic marathon times (women's race)



Wide neural networks are Gaussian processes

One hidden layer of p neurons and random weights:

$$f(x) = b + \frac{1}{\sqrt{p}} \sum_{j=1}^p a_j \varphi(w_j^T x + b_j), \quad a_j, w_j, b_j \stackrel{\text{iid}}{\sim} \text{Gaussian}$$

- For fixed x , the p terms $a_j \varphi(w_j^T x + b_j)$ are iid with finite variance
- Central limit theorem: as $p \rightarrow \infty$, $f(x)$ becomes Gaussian
- Same for any finite set of inputs: $(f(x_1), \dots, f(x_n))$ is *jointly Gaussian*

An infinitely wide network is a draw from a Gaussian process!

The neural network kernel (NNGP)

Covariance of the limiting process is a Mercer kernel:

$$K(x, x') = \text{Cov}(f(x), f(x')) = \sigma_b^2 + \sigma_a^2 \mathbb{E}_{w,b} [\varphi(w^T x + b) \varphi(w^T x' + b)]$$

- Architecture + weight distribution *determine the kernel*
- ReLU activations give the “arc-cosine kernel,” which depends on the angle between x and x'
- Deep networks: apply the argument layer by layer

Bayesian neural nets = GP regression

Model: $Y_i = m(X_i) + \epsilon_i$, $\epsilon_i \sim N(0, \sigma^2)$, with prior m = infinitely wide random network

Then the posterior over functions is just GP regression with the NNGP kernel. Predictive mean at a new point x :

$$\hat{m}(x) = \mathbb{E}(m(x) | Y) = k(x)^T (\mathbb{K} + \sigma^2 I)^{-1} Y$$

where $\mathbb{K}_{ij} = K(X_i, X_j)$ and $k(x) = (K(x, X_1), \dots, K(x, X_n))^T$

- Exact Bayesian inference over (infinitely many) weights reduces to an $n \times n$ linear system
- Comes with uncertainty: the predictive variance $K(x, x) - k(x)^T (\mathbb{K} + \sigma^2 I)^{-1} k(x)$

Two kernels: NNGP vs. NTK

	NNGP	NTK
Describes	network at <i>initialization</i>	network after <i>gradient descent</i>
Kernel	$\text{Cov}(f(x), f(x'))$	$\nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0)$
Prediction	Bayesian posterior mean	kernel (ridgeless) regression

- Wide nets barely move from θ_0 during training, so they behave like the linear model in features $\nabla_{\theta} f(x; \theta_0)$
- Trained output is an affine function of the random initialization $\Rightarrow$ *also a Gaussian process*, but generally *not* the Bayesian posterior
- Training only the last layer: $\text{NTK} = \text{NNGP}$

∞ -width limit theory: Strengths and limitations

Explains:

- Why heavily overparameterized networks train to zero error
- Connection to minimum-norm interpolation and double descent
- Lets us use kernel and GP tools to analyze deep learning

Misses:

- The kernel is fixed before seeing data: *no feature learning*
- Real (finite) networks typically outperform NNGP/NTK kernels
- Learning representations is what makes deep learning work